

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Computer Vision with OpenCV **COURSE CODE:**
DSA02

COURSE OUTCOME COVERED

***CO4:** Evaluate recognition and learning methods including machine learning and deep learning models (CNNs, GANs, SVM, HMM, etc.) for vision tasks.. (BL5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Total Marks:50

Annexure A

Comparative Analysis

Code of Conduct:

I B.Deepthi-192424269 (Name / Reg No) certifies that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the student:B.Deepthi

RUBRICS FOR CALCULATION

1.CNNs vs. Vision Transformers (ViT) for Object Detection Scenario

A self-driving car company needs to detect rare urban obstacles such as traffic cones and animals in real time using a limited labeled dataset.

Question 1: Compare CNNs and Vision Transformers

Factor	CNNs	Vision Transformers (ViT)
Feature extraction	Excellent at extracting local features such as edges, corners, textures and shapes	Better at learning about global relationships and long-range context between image regions
Local vs. global context	CNNs naturally focus on local regions using convolution filters; larger context requires deeper layers or larger kernels	Self-attention directly models relationships between distant image patches
Small/sparse objects	Generally effective because multi-scale CNN architectures such as FPN can preserve fine spatial details	Can capture global context well, but small-object detection may require high-resolution inputs and suitable attention mechanisms
Training with small datasets	Usually performs well with limited data, especially with pretrained models	Typically requires more data to train effectively from scratch
Computational complexity	Convolution operations are relatively efficient and highly optimized on embedded hardware	Self-attention can be computationally expensive, especially for high-resolution images
Inference speed	Generally faster and more suitable for real-time embedded applications	Standard ViT can be slower, although lightweight/efficient ViT have improved significantly
Memory requirement	Usually lower and more predictable	Can require higher memory because of attention operations
Embedded deployment	Very suitable	Possible with lightweight models, but standard ViT are more demanding

CNN Feature Extraction

CNNs process an image through convolution filters. The early layers learn features such as:
Edges → textures → shapes → object parts → complete objects

This makes CNNs particularly effective for detecting small local patterns such as the shape and color of a traffic cone.

ViT Feature Extraction:

A Vision Transformer divides an image into patches and uses self-attention to understand relationships between different patches.

For example, when detecting an animal, a ViT can consider the relationship between the animal's body, background, road and surrounding objects, providing stronger global context.

Conclusion

CNNs and Vision Transformers (ViTs) both offer strong capabilities for object detection, but they have different strengths. **CNNs** are efficient, computationally lighter, and effective at capturing local spatial features, making them suitable for real-time and resource-constrained applications.

ViTs, on the other hand, use self-attention to capture global relationships within an image and can achieve excellent detection accuracy, particularly when trained on large datasets.

Overall, **CNN-based detectors remain a practical choice for speed and efficiency, while ViT-based approaches are promising for achieving higher accuracy and better global feature understanding.** Hybrid CNN–ViT architectures can further combine the advantages of both approaches.

2: Which method is better suited for real-time deployment?

For this scenario, CNNs are generally better suited for real-time deployment, especially when the system uses embedded hardware in a self-driving vehicle.

Reasons

1. Faster inference

- CNNs are computationally efficient.
- They are well optimized on GPUs, NPUs and automotive embedded processors.

2. Lower computational requirements

- CNNs usually require less memory and computation than standard ViTs.

3. Better performance with limited data

- CNNs have strong inductive biases such as locality and translation equivariance.
- Therefore, they can learn useful visual features with fewer training examples.

4. Strong small-object detection

- CNN-based object detectors can use feature pyramids to detect objects at different scales.
- This is useful for distant traffic cones, animals and pedestrian
- **Role of Transfer Learning**

Transfer learning can significantly improve performance when labeled data is limited.

A CNN or ViT can first be pretrained on a large dataset such as ImageNet or a large object-detection dataset. The pretrained model already understands general visual features.

The model can then be fine-tuned using the company's smaller urban-obstacle dataset.

For example:

Large dataset → Pretrained model → Fine-tuning → Traffic cone/animal detector

This reduces training time and improves accuracy.

Role of Data Augmentation

Data augmentation can create variations of the available training images.

Useful techniques include:

- Rotation
- Horizontal flipping
- Scaling
- Cropping
- Brightness changes
- Contrast changes
- Blur
- Noise addition
- Weather simulation
- Occlusion

For autonomous driving, augmentation can simulate night-time conditions, rain, shadows, different viewing angles and partial occlusion.

Conclusion

For a self-driving car with limited labeled data and strict real-time requirements, a CNN-based detector is generally the safer choice. However, lightweight Vision Transformers or CNN–Transformer hybrid architectures can be considered when greater global-context understanding is required and sufficient computational resources are available.

2.PCA-Based Shape Priors vs. Graph-Based Learning for 3D Shape Correspondence Scenario

An animation studio needs to align 3D human face meshes for morphing. The meshes have different topology and vertex densities.

Question 1: Compare PCA-based shape priors and graph-based learning

Factor	PCA-Based Shape Priors	Graph-Based Learning
Basic idea	Represents shapes using a low-dimensional statistical model	Represents a 3D mesh as a graph and learns relationships between vertices
Non-rigid deformation	Handles common deformations if they are represented in the training data	Usually better at complex and non-rigid deformations
Varying mesh resolution	Sensitive to vertex correspondence and representation	More naturally handles irregular mesh structures
Different topology	Difficult because PCA normally requires aligned/ corresponding vertices	More flexible for irregular mesh structures and topology variations
Noise robustness	Can suppress noise because PCA retains dominant variations	Can learn robust features, but performance depends on graph architecture and training
Implementation	Relatively simple	More complex
Training requirement	Requires a collection of aligned training shapes	Usually requires training data and a suitable graph neural network
Computational cost	Low after PCA model construction	Higher because graph operations and neural-network training are required
Interpretability	High; principal components represent major shape variations	Lower; learned graph features are more difficult to interpret
Morphing applications	Good for controlled and similar face datasets	Better for complex real-world mesh variations

PCA-Based Shape Priors

Principal Component Analysis (PCA) represents a shape using a smaller number of principal components.

For example, facial shapes can be represented as:

Mean Face + Shape Variations

The major variations might correspond to:

- Face width
- Nose length
- Jaw shape
- Eye position
- Cheek structure

PCA reduces the dimensionality of the 3D mesh and captures the most important variations.

Advantages

- Simple to implement
- Computationally efficient
- Easy to interpret
- Effective when meshes have similar topology
- Good for controlled facial datasets

Disadvantages

- Requires consistent vertex correspondence
- Not naturally suited to different mesh topologies
- Sensitive to changes in vertex density
- May struggle with highly nonlinear deformations

Graph-Based Learning

In graph-based learning, a 3D mesh is represented as a graph:

- Vertices → graph nodes
- Mesh edges → graph connections
- Geometric information → node/edge features

A Graph Neural Network (GNN) can then learn relationships between neighboring and distant parts of the mesh.

This makes graph-based methods particularly useful when facial meshes have different structures or irregular sampling.

Advantages

- Better suited to irregular mesh structures
- Can learn complex nonlinear relationships
- More flexible for non-rigid deformation
- Can incorporate local geometric relationships
- More adaptable to varying mesh structures

Disadvantages

- More difficult to implement
- Requires more computational resources
- Usually requires substantial training data
- Model training can be more complex than PCA

Question 2: Which approach is better for this application?

For the animation studio's application, graph-based learning would generally provide better performance, especially because the meshes have varying topology and vertex density.

Why?

The major challenge is that PCA generally assumes that corresponding vertices represent corresponding points on different faces.

For example:

Mesh A: 10,000 vertices

Mesh B: 15,000 vertices

The vertices may not directly correspond to each other.

PCA becomes difficult to apply directly because the shape vectors must have a consistent representation.

Graph-based methods are more flexible because they operate on the **relationships between mesh vertices**, rather than relying exclusively on a fixed vertex ordering.

For non-rigid deformation

Human faces can change significantly due to

- Smiling
- Frowning
- Opening the mouth
- Raising eyebrows
- Moving the jaw
- Changing facial expressions

These transformations are often nonlinear.

Graph-based learning can learn these complex relationships better than a simple linear PCA

model.

For noise

3D scans may contain:

- Missing regions
- Measurement noise
- Irregular vertices
- Different mesh densities

A properly trained graph-based model can learn robust geometric representations and reduce the influence of noisy local information